

## Día D — sábado, instrucciones en caliente

El profesor calificará **evolución de un producto que ya funciona**, no una app construida en vivo. Esta hoja cubre las dos cosas que decide la nota: la **línea base desplegada** que hay que tener antes, y los **10 minutos en vivo** cuando dicte las dos Historias de Usuario nuevas. Los comandos se copian tal cual; lo **resaltado** es lo único que se cambia.

### Sin esto no hay nota, y no se puede improvisar el sábado

La regla 1 del profesor: «*al menos un flujo funcional completo Frontend → Backend/API → Base de Datos → Backend → Frontend*», desplegado. Y descalifica explícitamente *frontend con datos simulados*, *APIs no consumidas desde el frontend* y *datos solo en memoria*. Hoy el repo tiene la fase 1: el cascarón, los mocks y 72 pruebas. **Las rutas de API, la base de datos y el despliegue todavía no existen**. Las fases 2 a 5 son trabajo de **antes** del sábado, no del Día D.

### La línea base — cerrada el viernes, no el sábado

| ✓                        | Quién    | Qué tiene que estar cierto                                                                                                                                                                                                                                  |
|--------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | Mateo    | fase 2 mergeada en main — las pantallas, hablando por <code>fetch</code> .                                                                                                                                                                                  |
| <input type="checkbox"/> | Johan    | fase 3 mergeada — <code>lib/db.js</code> , <code>lib/higgsfield.js</code> y las cuatro rutas bajo <code>app/api/</code> .                                                                                                                                   |
| <input type="checkbox"/> | Tomás    | <b>T030 primero, no último</b> . Las ocho llaves reales (Clerk, MongoDB Atlas, Vercel Blob, Higgsfield) en su <code>.env.local</code> y en <code>Project Settings → Environment Variables</code> de Vercel. Sin esto lo demás no se puede probar de verdad. |
| <input type="checkbox"/> | Santiago | fase 5: <code>merge</code> , <code>npm test</code> , <code>npm run build</code> y <b>T039 — desplegar en Vercel</b> . La URL pública escrita aquí: <a href="https://ia-generator-openspec.vercel.app">https://ia-generator-openspec.vercel.app</a>          |
| <input type="checkbox"/> | Santiago | <code>npm run smoke</code> <a href="https://ia-generator-openspec.vercel.app">https://ia-generator-openspec.vercel.app</a> en verde. Prueba las tres capas contra el despliegue real, no contra <code>localhost</code> .                                    |
| <input type="checkbox"/> | Todos    | El recorrido a mano en el celular, sobre la URL pública: <code>entrar</code> → <code>foto</code> → <code>imagen</code> → <code>video</code> → <code>descargar</code> . Eso es el «flujo completo» que pide la regla 1.                                      |

Si el viernes falta algo, se recorta alcance, no calidad: la regla 9 del profesor dice que *una funcionalidad pequeña completamente integrada vale más que una enorme que no funciona*. El mínimo que cumple la regla 1 son las fases 2 y 3 con llaves reales y desplegadas — el dashboard (fase 4) puede quedar para después.

### 20 minutos antes de que empiece la clase — los cinco

Cada uno, en su máquina, dentro de la carpeta `speckit-ai-generator`:

```
git checkout main
git pull origin main
npm install
npm test
```

Tiene que decir `0 failed`. Si sale rojo, avisa al grupo **ahora**. Deja tu agente abierto en esa carpeta. Nadie usa carpeta de demo: se trabaja sobre el repo real.

- Santiago** — `npm run smoke` <https://ia-generator-openspec.vercel.app> y abre la URL en el celular. Deja el panel de Vercel abierto en una pestaña.
- Tomás** — confirma que su `.env.local` con las ocho llaves sigue en su máquina.
- Sergio** — abre `spec.md` y `tasks.md` en pantalla, listos para proyectar.
- Todos** — hotspot del celular encendido, por si falla el wifi del salón.
- Todos** — `git log --graph --oneline -15` a mano: el grafo de ramas es la prueba visual de «una historia, una rama».

## La secuencia en vivo — 10 minutos

Cada quien corre el ciclo completo en su propia máquina y sobre su propio archivo de spec: Historia → Spec → Plan → Tasks → Implementación → Pruebas → Commit. Nadie mira a otro operar la IA.

### 1 Sergio registra la necesidad y reparte

≈ 2 min · rama `main`

Proyecta la pantalla. Deja la rama limpia y pega el prompt con las historias literales que dictó el profesor:

```
git checkout main
git pull origin main
```

```
/speckit-specify Agrega dos User Stories nuevas al final de la seccion User Scenarios de
specs/001-ai-media-generator/spec.md, numeradas User Story 5 y User Story 6.
HU 5: "historia literal que dictó el profesor"
HU 6: "segunda historia literal"
Cada una con sus Acceptance Scenarios numerados en formato Given/When/Then.
NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
NO toques plan.md ni tasks.md. Solo edita ese spec.md que ya existe.
```

```
git add specs/001-ai-media-generator/spec.md
git commit -m "HU en vivo: User Stories 5 y 6 dictadas en clase"
git push origin main
```

Ese commit, con la hora de hoy, es la prueba de que la necesidad entró en vivo. Es el primer eslabón de la trazabilidad.

**Cómo repartir.** Mira los Acceptance Scenarios y decide qué archivos toca cada uno. La regla es **un archivo, un dueño** — nunca dos personas sobre el mismo archivo:

Toca archivos **nuevos**  
pantalla o ruta que no existe todavía

Los tres en paralelo: **Mateo** la pantalla, **Johan** la ruta de API, **Tomás** la consulta o el reporte.

Toca una **pantalla existente + una ruta de API**  
son archivos distintos, se reparte solo

Dos personas: una la pantalla ( `app/.../page.jsx` ), otra la ruta ( `app/api/...` ).

Toca **un solo archivo**  
no hay forma de partirlo sin que dos lo editen

**Una sola persona** hace esa HU completa. Los otros toman la otra HU.

**Di en voz alta tres cosas antes de que nadie arranque:** qué le toca a cada uno, qué archivos son suyos, y **el nombre de su rama** — una palabra que describa la funcionalidad, no la persona: `feature/categorias`, `feature/exportar`, `feature/auditoria`. Si sobra trabajo, **Santiago** toma una parte con el mismo bloque del paso 2 y hace el merge después.

### 2 Mateo, Johan y Tomás corren el ciclo completo, cada uno en su máquina

≈ 6 min

**2.1 — Tu rama.** Cada uno con el nombre que dijo Sergio:

#### MATEO

```
git checkout main
git pull origin main
git checkout -b feature/_____
```

#### JOHAN

```
git checkout main
git pull origin main
git checkout -b feature/_____
```

#### TOMÁS

```
git checkout main
git pull origin main
git checkout -b feature/_____
```

**2.2 — Tu Spec, tu Plan y tus Tasks.** Pégame esto a tu agente. Escribe **tu nombre** en la ruta: un archivo por persona, así dos specs nunca chocan.

Escribe `specs/001-ai-media-generator/hu-viva/tu-nombre.md`, y solo ese archivo.

Historia: "`la parte de la HU que me tocó, literal`"

El archivo lleva cuatro secciones:

1. Historia — la frase de arriba, textual.
2. Criterios de Aceptacion — numerados CA-1, CA-2, CA-3, en formato Dado / Cuando / Entonces.
3. Plan — la lista exacta de archivos que vas a crear o modificar, y por que cada uno.
4. Tasks — numeradas, y cada una dice que CA cubre.

Reglas: no toques `spec.md`, `plan.md` ni `tasks.md`. Si para cumplir la Historia necesitas modificar un archivo que ya existe y no esta en tu Plan, parate y dimelo antes de escribir.  
Persistencia real: si la Historia guarda datos, van a MongoDB por `lib/db.js`, nunca en memoria.

**2.3 — Implementa, test-first.** Lee el Plan en voz alta antes de lanzar esto — es tu último filtro contra un archivo que sea de otro:

Implementa `specs/001-ai-media-generator/hu-viva/tu-nombre.md`, test-first:  
por cada Criterio de Aceptacion, primero la prueba que falla y despues el codigo que la pasa.

Nombra cada prueba empezando por su criterio: `it("CA-2: ...")`.  
Toca unicamente los archivos de la seccion Plan de ese documento. Ninguno mas.  
Al terminar corre `npm test` y dejalo en verde.

Ese `it("CA-2: ...")` no es cosmético: es la respuesta literal cuando el profesor pregunte «¿qué criterio de aceptación demuestra esta prueba?».

**2.4 — Pruebas, commit y push.** El `npm test` lo corres tú, con tus ojos, antes de que la rama salga de tu máquina:

```
npm test
git add -A
git commit -m "feat: la historia en cinco palabras (CA-1 a CA-N)"
git push -u origin feature/_____
```

Si `npm test` sale rojo, no hagas push. Dilo en voz alta y pídele al agente: «la prueba CA-2 está fallando, arréglala y vuelve a correr `npm test`». Que el método atrape el error delante del profesor juega a favor. Cuando tu `git push` termine, anúncialo: «categorias arriba, tres pruebas en verde» — **Santiago** no arranca hasta oír los tres.

### 3 Santiago integra, prueba, construye y despliega

≈ 2 min · rama `main`

Solo cuando los tres hayan dicho que subieron:

```
git checkout main
git pull origin main
git fetch origin
```

Un merge por línea, y con `--no-ff` para que cada historia quede como una rama visible en el grafo. Espera a que termine cada uno antes de escribir el siguiente:

```
git merge --no-ff origin/feature/_____
git merge --no-ff origin/feature/_____
git merge --no-ff origin/feature/_____
```

Con los tres adentro, la cadena completa antes de subir:

```
npm test
npm run build
git push origin main
```

Ese push a `main` dispara el despliegue en Vercel por sí solo, y CI vuelve a correr lint, pruebas y build sobre el merge. Nadie corre un comando de deploy a mano.

Mientras Vercel construye, proyecta el grafo — es la respuesta visual a «¿cómo evitan los conflictos?»:

```
git log --graph --oneline -15
```

### 4 Todos validan sobre el despliegue, no sobre localhost

≈ 1 min

1. **Santiago** proyecta el panel de Vercel y espera el check verde del deploy (1–2 min).
2. Con el deploy en verde: `npm run smoke https://ia-generator-openspec.vercel.app` — tres capas comprobadas en dos segundos, delante del profesor.
3. Abre `https://ia-generator-openspec.vercel.app` en el celular y recorre la Historia nueva de punta a punta.

Si el deploy todavía no termina, **Santiago** corre `npm run dev` y muestra la app en `http://localhost:3000` mientras tanto — pero **vuelve a la URL pública** apenas el deploy esté verde: la regla 1 pide despliegue, y una demo que solo corre en local no cuenta.

## Trazabilidad — lo que va a preguntar, y quién contesta

El profesor anunció dos preguntas textuales. Estas son las respuestas, con el archivo que hay que abrir en pantalla. Cada uno contesta por lo suyo; nadie responde por otro.

| Pregunta                                             | Contesta | Qué abre y qué dice                                                                                                                                                                                                                      |
|------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| «¿Qué parte del Spec produjo esta implementación?»   | El dueño | Abre <code>specs/001-ai-media-generator/hu-viva/tu-nombre.md</code> . Señala el <b>CA-N</b> , la <b>Task</b> que lo cubre y el archivo que la Task nombra en el Plan. Los tres están en la misma pantalla.                               |
| «¿Qué criterio de aceptación demuestra esta prueba?» | El dueño | El nombre de la prueba lo dice. Demuéstralo corriendo solo ese criterio:<br><code>npm test -- -t "CA-2"</code>                                                                                                                           |
| «¿Dónde está la necesidad original?»                 | Sergio   | <code>spec.md</code> , User Stories 5 y 6 — las que usted dictó. Y el commit que las metió, con la hora de hoy: <code>git log -1 --format='%h %cd %s' -- specs/001-ai-media-generator/spec.md</code>                                     |
| «¿Y esto no lo tenían hecho de antes?»               | Sergio   | <code>git log --graph --oneline -15</code> . El commit de la HU y los tres <code>feature/...</code> tienen la hora de hoy; todo lo anterior es de la semana pasada, y está en GitHub con su fecha.                                       |
| «¿Cómo evitan los conflictos?»                       | Sergio   | Cada spec declara sus archivos en la sección <b>Plan</b> , y ninguno aparece en dos. Lo que una parte necesita de otra la llama por la firma acordada en <code>plan.md</code> , sección <b>Contracts</b> . Por eso nadie espera a nadie. |
| «¿Dónde se guardan los datos?»                       | Johan    | <code>lib/db.js</code> → MongoDB Atlas, colección <code>generations</code> . Nada en memoria. Lo prueba recargando la página en el celular: el dato sigue ahí.                                                                           |
| «¿Esto está desplegado o corre en su máquina?»       | Santiago | El panel de Vercel con el deploy de hace dos minutos, y <code>npm run smoke</code> apuntando a la URL pública. El celular está sobre datos móviles, no sobre el wifi del salón.                                                          |

## Si algo se rompe

| Qué pasó                                         | Quién lo arregla | Qué hace, exactamente                                                                                                                                                                                                   |
|--------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conflicto en el merge                            | Santiago         | En <code>package.json</code> : dejar las dependencias de los dos lados. En cualquier <code>lib/*.js</code> : dejar la implementación real y borrar el stub. Después <code>git add -A</code> y <code>git commit</code> . |
| El merge se enredó                               | Santiago         | <code>git merge --abort</code> y volver a intentar solo con esa rama. No pierde nada de lo ya mergeado.                                                                                                                 |
| El agente toca archivos de otro                  | El dueño         | Pararlo y decirle: « <i>revierte eso, solo puedes tocar los archivos de la sección Plan de mi spec</i> ».                                                                                                               |
| El agente se pierde o se contradice              | Sergio           | <code>/speckit-analyze</code> — no cambia nada, muestra qué se contradice entre spec, plan y tickets.                                                                                                                   |
| <code>npm test</code> en rojo                    | Quien lo corrió  | No hacer push. Decirle al agente qué CA falla y que lo arregle. Volver a correr <code>npm test</code> .                                                                                                                 |
| <code>npm run build</code> en rojo tras el merge | Santiago         | No hacer push a <code>main</code> : un build roto tumba el despliegue. Arreglar solo lo que rompió el merge, volver a construir.                                                                                        |
| El deploy de Vercel falla                        | Santiago         | Abrir el log del deploy. Casi siempre es una variable de entorno faltante: se agrega en <i>Project Settings</i> y se hace <i>Redeploy</i> . Mientras tanto, <code>npm run dev</code> .                                  |
| Se cayó el wifi                                  | Todos            | Hotspot del celular. Las pruebas corren sin llaves y sin internet gracias a los dobles del T008; el deploy y la demo sí necesitan red.                                                                                  |

## Las seis reglas que no se rompen

1. Es `/speckit-implement`, **no** `/implement`. Y nunca a secas: sin argumento el agente construye todo el backlog.
2. **Primero el spec, después el código.** Código generado sin especificación previa no demuestra SDD y no cuenta — es la regla 6 del profesor.
3. **Una prueba por criterio de aceptación**, nombrada con su `CA-N`, y escrita antes del código.
4. Nadie escribe fuera de la sección **Plan** de su propio spec. Un archivo, un dueño.
5. Los merges van **uno por uno** y con `--no-ff`. `git merge A B C` se cancela entero al primer conflicto.
6. Nadie hace `git push origin main` en vivo salvo **Sergio** en el paso 1 y **Santiago** en el paso 3. Los demás empujan solo a su rama `feature/...`.